

BiteQuest Frontend Product Design Document

Owner: your friend | Contract: v1.0 | Date: October 3 2026

Purpose and outcome

Build a mobile-first restaurant exploration app where diners discover restaurants on a map, collect them after a purchase, earn platform-funded rewards, and use that value at another restaurant. BiteQuest is a working name. The visual experience borrows location-based collection mechanics, with original branding and assets.

This document is the frontend owner's implementation brief. The backend owner follows 02-backend-pdd.md. Both use shared/contracts.ts and shared/fixtures.json. The frontend must work against an in-memory mock adapter before the live APIs exist; swapping adapters must not change any screen.

Product decisions

- XP is permanent game progression. Reward credits are portable monetary value, stored in USD cents and displayed as dollars. One credit unit is one cent; XP is never deducted during redemption.
- The platform funds the initial rewards. Restaurants do not need to onboard or subsidize rewards for this demo.
- A qualifying visit earns 50 cents. Rarity changes first-discovery XP, not monetary reward: common 150, rare 400, epic 800, legendary 1200.
- $\text{Level} = 1 + \text{floor}(\text{totalXp} / 1000)$. Show progress toward the next 1000 XP. Only the first discovery of a restaurant earns XP.
- Purchase confirmation and reward settlement are simulated and visibly labeled. Paying a restaurant normally does not automatically let our app discount its checkout.

Ownership and scope

Area	Frontend owner	Backend owner
Experience	Map, panels, navigation, animation	Nearby data and eligibility
Visits	Location capture and confirmation UI	Validation and atomic award
Wallet	Display balance, history, redemption form	Ledger, limits, atomic debit
Adventure	Form, streamed progress, route display	Agent, retrieval, route validation
Shared files	Consume frozen types and fixtures	Own contract and seed definitions

MVP screens: Explore, restaurant details, check-in, discovery celebration, Collection, Wallet, and Adventure. Defer merchant dashboards, social features, reviews, account settings, receipt OCR, real payment processing, H3 zones, and 3D characters.

Frontend libraries

Use Next.js App Router and TypeScript, Tailwind, shadcn primitives, react-map-gl with mapbox-gl, Motion, canvas-confetti, TanStack Query, and Zustand. Use a Dialog styled as a bottom panel for mobile; a draggable sheet is optional. Keep the map in a client-only component. Pin exact compatible versions in the shared lockfile after the scaffold installs successfully.

Explore and restaurant details

The map is the home screen. It fills the space behind a compact header, category controls, and bottom navigation. The header displays level and reward balance. Bottom navigation contains Explore, Collection, and Wallet; Adventure is a prominent action on Explore.

Use a muted, playful basemap with simplified labels, visible roads and water, and a modest camera tilt. Preserve map attribution and readable orientation. Do not make a custom Mapbox Studio style a launch dependency; start from a supported style, then tune it if time remains.

Restaurant markers show a cuisine icon, rarity color, and a collected checkmark. Selected markers have a larger outline. Rarity must also appear as text or an icon so color is not the only signal. Render 15-20 markers initially; deck.gl is outside MVP scope.

Opening a restaurant updates one selectedRestaurantId and moves the camera only enough to keep the marker above the panel. Dismissing the panel returns to the same map position. Filters operate on the fetched catalog; a filter with no results shows a clear reset action.

Restaurant panel contents

- Name, cuisine, rarity, first-discovery XP, and reward amount.
- Estimated meal price labeled as an estimate; distance labeled as straight-line distance.
- Collected or undiscovered status; reward eligibility supplied by the backend.
- Availability displayed only when known. Do not invent live ratings, opening hours, photos, or restaurant participation.
- Actions: Navigate, Check in, and Use rewards. Navigation opens an external map link using coordinates.
- An original cuisine illustration when imageUrl is null. Synthetic demo venues must be visibly identified as demo data.

Location and browsing

Ask for browser geolocation after a user action such as Use my location. Capture latitude, longitude, accuracyMeters, and capturedAt. Start browsing around the SF demo center if permission is denied. This fallback center does not count as verified physical presence.

Refresh nearby restaurants on an explicit Search this area action or recentering, with radiusMeters = 3000. Do not query on every camera frame. The response's center is the reference for distanceMeters. Marker positions use coordinates; route arrays use [longitude, latitude].

Responsive layout

Viewport	Layout
360-767 px	Full map, bottom panel up to 70% viewport height, three-tab navigation
768-1023 px	Map with a floating panel up to 420 px wide
1024 px and above	Map with a 380 px side panel; collection and wallet use a grid

Use 44 px minimum touch targets, safe-area padding, and scrollable panel content. Test at 360 x 800, 390 x 844, 768 x 1024, and 1440 x 900. The map or bottom navigation must never trap the purchase confirmation button.

Check in and discovery

Check-in flow: select restaurant, capture current location, enter the demo bill amount, confirm the simulated purchase, submit, then display the server's award. Show a persistent Demo purchase label. The server requires a bill of at least \$5 and a location within 100 meters, no older than 60 seconds, with accuracy no worse than 100 meters.

Provide a clearly labeled Simulate location at this restaurant action only when server bootstrap reports verificationMode = demo. A frontend query parameter cannot enable this mode. Simulated coordinates must not be presented as real GPS verification.

The purchase form generates one receiptId and one Idempotency-Key for its submission. Keep both on ambiguous network failures and retry the same request. A new user action gets new IDs. Disable duplicate taps while pending, but rely on backend idempotency for correctness.

Mutation state machine

Idle -> locating -> confirming -> submitting -> success or error. Only submitting invokes the visit API. On confirmed success, replace the profile cache from the response, mark the venue collected, and invalidate nearby, collection, and wallet queries. Never award money optimistically.

If the response is lost after submission, show Retry confirmation and reuse the original key. Repeated success for the same visitId must not trigger a second celebration. On a validation error, show the specific correction and keep the user's bill amount.

Celebration

Show a restaurant collection card with rarity, xpAwarded, and creditAwardedCents. Animate the card with Motion, then update the XP bar and wallet counter from authoritative values. Confetti fires on first discovery or level-up only. Respect prefers-reduced-motion and provide a Continue button immediately.

A repeated visit can earn credits on a later eligible day, but gives 0 discovery XP and uses a compact reward confirmation rather than a new-collection animation. The server enforces one earning visit per restaurant per SF calendar day and a \$2 daily earning cap across all restaurants.

Collection

Display discovered restaurants as a responsive card grid with cuisine, rarity, and discovery date. Include an empty state pointing back to Explore, a count, and cuisine filters. Collected cards open the same restaurant panel. A newly added card must appear without a reload.

Error copy

Code	User-facing meaning
LOCATION_TOO_FAR	Move closer or use the labeled demo location action
LOCATION_STALE or LOCATION_INACCURATE	Refresh your location and try again
ALREADY_EARNED_TODAY	This restaurant has already earned today's reward
DAILY_REWARD_CAP	Today's \$2 reward limit has been reached
PURCHASE_TOO_SMALL	Enter a demo bill of at least \$5
UNAUTHENTICATED	Reload to establish a demo session
IDEMPOTENCY_CONFLICT	Start a new confirmation for the changed request

Wallet and portable redemption

Wallet shows an available balance in dollars and a chronological history with restaurant, date, and signed amount. Earn entries say Platform-funded reward. Redeem entries say Demo redemption. Include rewardFunding = platform in the summary copy.

Use integer cents internally. Format using Intl.NumberFormat. Parse typed amounts as decimal strings into cents; do not use floating-point dollars for ledger math. Users cannot change XP or balances through a UI store.

Redemption journey

Open Use rewards from a different restaurant's panel or choose a target restaurant from Wallet. Enter the total bill and the requested reward amount. Preview the bill, reward contribution, remaining personal contribution, and wallet balance after redemption. The preview is an estimate; the final confirmation uses the server response.

The MVP allows a redemption from 1 cent up to the lesser of wallet balance, bill amount, and \$5. The target must be in the catalog. Server rules require at least one earning visit at a different restaurant to demonstrate portability; there is no source-specific credit tracking.

Confirm with a new Idempotency-Key and demoReceiptId. On success, display settlementStatus = simulated, reward amount consumed, and the new wallet balance. Use this message: Demo complete. Your reward balance was applied in the simulation. This is not a payment to the restaurant.

Redemption does not itself collect the restaurant, earn XP, or award more credits. If the user also checks in there, that is a separate qualifying purchase action.

Wallet states

- Loading: skeleton balance and history rows.
- Empty: \$0.00 and a link to Explore.
- Pending redemption: disable Confirm and show progress.
- Insufficient balance: preserve the entered bill; ask the user to lower the reward amount.

- Network timeout: retry the same redemption key, never issue a new debit automatically.
- Confirmed success: replace summary cache, invalidate wallet query, close the form after user acknowledgement.
- Database or API unavailable: keep a readable error state; never silently switch the real adapter to a local wallet.

Adventure experience

Adventure accepts `maxSpendCents`, `maxDurationMinutes`, `maxStops` from 1 to 3, preferences, and `onlyUndiscovered`. Default to a \$30 budget, 90 minutes, at most 2 stops, and undiscovered venues.

Submit to the streaming endpoint with `fetch` and an `AbortController`. Display concise progress messages supplied by the API. Never surface hidden chain-of-thought or raw provider logs. A Cancel button stops the request. A failed connection can restart planning because planning has no financial side effects.

Show the completed plan as ordered venue cards plus a map line. Each stop shows a short reason, estimated meal cost, XP, and potential reward. Totals are server-calculated. Display `routeKind` = estimate as Approximate walking route; do not imply turn-by-turn directions. Start adventure selects the first stop and draws the plan; it does not issue rewards.

Frontend engineering plan

Keep all application data behind a `BiteQuestApi` interface. Implement `MockApi` and `HttpApi` with identical return types. Use `NEXT_PUBLIC_API_MODE` = mock or live at startup. This is a developer switch; users cannot silently fall back into a simulated wallet after an API failure.

The mock adapter starts from `shared/fixtures.json`, keeps state in memory, validates reward eligibility, and updates its summary, ledger, and collection consistently. It supports an optional reset for development. Refreshing the page resets mock state; live state must persist in the database.

File ownership

Frontend-owned paths	Purpose
<code>app/page.tsx</code> , <code>app/collection/page.tsx</code> , <code>app/wallet/page.tsx</code>	Screen composition
<code>components/map/</code> , <code>components/restaurants/</code>	Map and venue panels
<code>components/collection/</code> , <code>components/wallet/</code> , <code>components/adventure/</code>	Feature views
<code>lib/client/</code> , <code>lib/mock/</code> , <code>hooks/</code> , <code>stores/</code>	API adapters, queries, local UI state
<code>app/globals.css</code> , <code>public/</code>	Styling and original assets

Backend-owned paths are `app/api/v1/`, `lib/server/`, and `supabase/`. Both owners may propose contract edits, but only the backend owner applies them before both owners acknowledge the change. One person owns `package.json`, the lockfile, `app/layout.tsx`, and Next configuration during the scaffold.

State and integration

TanStack Query owns summary, nearby, collection, and wallet data. Zustand owns selectedRestaurantId, map camera, active tab or overlay, and the active adventure. Keys include the session identity and center/filter inputs where relevant. Clear session-scoped caches when a session changes.

Freeze endpoint paths, fields, error codes, and the fixtures before feature work. UI components never call Supabase or sponsors directly. Prefix money fields with their unit suffix Cents. Use a single same-origin API base /api/v1.

Build order and timeboxes

1. First 30 minutes together: scaffold repository, pin dependencies, install the contract, and confirm ownership.
2. Next 60 minutes: map, selected restaurant panel, navigation, and responsive layout against mocks.
3. Next 60 minutes: purchase form, celebration, collection, and wallet mutation flows.
4. Next 45 minutes: Adventure form, streaming progress, and route display.
5. Reserve 60 minutes together for live adapter integration, failure states, and demo rehearsal. Stop optional styling work when this window begins.

Acceptance checklist

- All main screens work at the four target viewports without horizontal overflow.
- A restaurant can be selected, collected, and viewed in Collection.
- A visit at A changes balance from \$0.00 to \$0.50; a 30-cent redemption at B leaves \$0.20.
- Duplicate taps and lost-response retries do not award or redeem twice.
- Denied location, offline API, insufficient balance, and empty collections have usable states.
- Keyboard focus, dialog dismissal, reduced motion, and visible labels work.
- Mock and live adapters use the same types; live reload retains server state.

Shared integration contract

Authoritative files: shared/contracts.ts and shared/fixtures.json. This summary is repeated in the backend PDD. Contract v1.0 uses camelCase JSON, integer cents, integer XP, ISO UTC timestamps, opaque string IDs, and nullable unknown values. Point objects contain latitude and longitude; route coordinates are [longitude, latitude].

Method and path under /api/v1	Contract
POST /session/demo	Establish cookie session; return SessionResponse
GET /me	Return Summary
GET /restaurants/nearby	latitude, longitude, radiusMeters; return NearbyResponse
GET /collection	Return CollectionResponse
GET /wallet	Return WalletResponse

Method and path under /api/v1	Contract
POST /visits	VisitRequest -> VisitResponse; requires Idempotency-Key
POST /redemptions	RedemptionRequest -> RedemptionResponse; requires Idempotency-Key
POST /adventures/stream	AdventureRequest -> SSE StreamEvent frames

REST success is the typed object directly, without an extra data wrapper. Errors use { error: { code, message, retryable, details? } }. Session creation returns 200; first successful visit or redemption returns 201; replay returns the same status and saved response. Reads return 200. Use 401 for no session, 403 for blocked origin or demo mode disabled, 404 for unknown restaurant, 409 for conflicting or ineligible state, 422 for malformed input or failed location/purchase validation, 429 for rate limiting, and 503 for temporary service failure.

Stream contract

Each SSE frame has event: progress, complete, or error, and data: JSON encoding the corresponding StreamEvent union. Parse a streamed fetch response, not EventSource, because this endpoint is POST. Support arbitrary network chunk boundaries and UTF-8 decoding. Exactly one terminal event is sent after streaming begins.

Progress includes stage and message. Complete includes plan. Error includes the shared error object. Authentication and input errors occur as normal JSON HTTP errors before SSE begins. A disconnected stream without a terminal event is incomplete and must never be treated as a finished plan.

Joint integration sequence

1. Merge the frontend shell and backend routes into the same Next.js app; preserve the frozen contract.
2. Bootstrap the demo cookie session before any authenticated read. Use credentials: same-origin and validate write origins server-side.
3. Switch the client adapter to live. Confirm the seed restaurant IDs match the fixture IDs.
4. Run visit at A, replay it, reload, and verify collection, XP, balance, and ledger agree.
5. Redeem at B, replay it, and verify one debit and the remaining balance.
6. Generate an adventure, cancel one, and demonstrate a labeled deterministic fallback when a provider fails.
7. Rehearse one 90-second demo with a fresh session. Label synthetic venues, demo purchases, and demo redemption visibly.

References and implementation notes

react-map-gl documentation: <https://visgl.github.io/react-map-gl/docs>

Motion documentation: <https://motion.dev/docs/react>

Mapbox Search Box API: <https://docs.mapbox.com/api/search/search-box/>

Mapbox Search Box results are temporary-use data. The MVP uses an app-authored synthetic catalog so collected venues can be persisted without making Search Box data the database seed. Any later live discovery integration needs a suitable data source or storage permission. Sponsor and API references were checked on October 3 2026; exact package versions are fixed during setup.